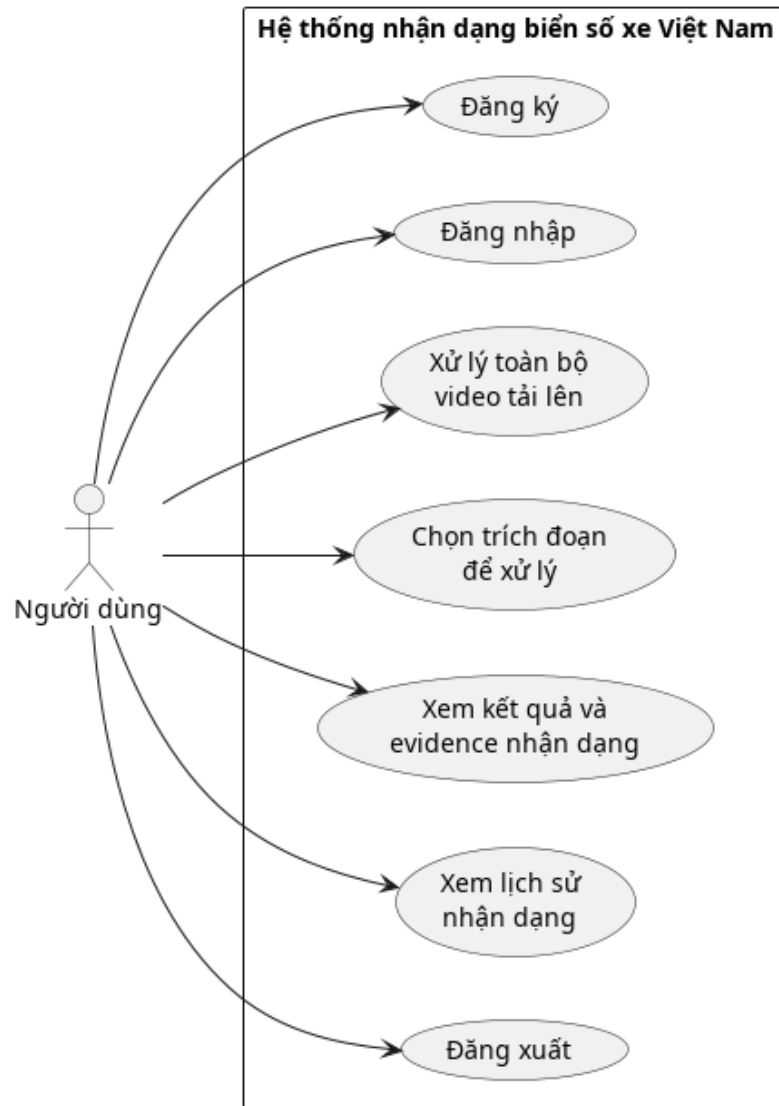


0.1 Yêu cầu và kiến trúc tổng quan

0.1.1 Sơ đồ use case



Hình 0.1: Sơ đồ use case tổng quan

Tác nhân tham gia Hệ thống có một tác nhân duy nhất là Người dùng đại diện cho bất kỳ cá nhân nào truy cập hệ thống qua giao diện web. Tác nhân này đóng hai vai trò tùy theo trạng thái xác thực: với các chức năng quản lý tài khoản, người dùng là khách chưa được xác thực; với các chức năng nghiệp vụ chính như xử lý video, phân tích sự kiện từ video, người dùng phải đã đăng nhập hợp lệ. Hệ thống không phân biệt vai trò theo nhóm quyền mà kiểm soát truy cập hoàn toàn dựa trên phiên đăng nhập.

Các use case chính Hệ thống có bảy use case chia thành hai nhóm chức năng.

Nhóm quản lý tài khoản gồm ba use case. *Đăng ký* cho phép người dùng tạo tài

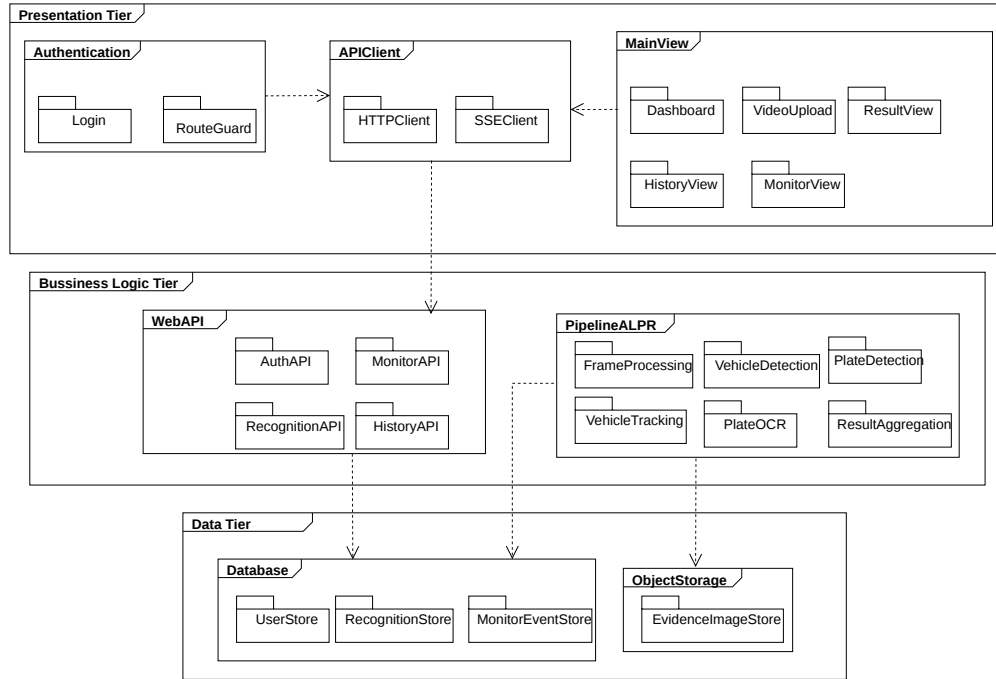
khoản mới; hệ thống xác thực dữ liệu đầu vào, băm mật khẩu và lưu thông tin vào cơ sở dữ liệu. *Đăng nhập* cho phép người dùng xác thực danh tính và nhận token bảo mật để truy cập các chức năng được bảo vệ. *Đăng xuất* hủy phiên làm việc hiện tại và đưa người dùng về trạng thái khách.

Nhóm nghiệp vụ nhận dạng gồm bốn use case, chỉ khả dụng sau khi đăng nhập. *Xử lý toàn bộ video tải lên* là luồng xử lý chính: người dùng tải lên một tệp video hoàn chỉnh, hệ thống chạy pipeline ALPR trên toàn bộ nội dung video và trả tiến độ, kết quả nhận dạng về giao diện theo thời gian thực. *Chọn trích đoạn để xử lý* phục vụ trường hợp người dùng chỉ cần phân tích một khoảng thời gian cụ thể từ nguồn camera trực tiếp hoặc từ video đã tải lên. Trong luồng này, người dùng chọn nguồn video, đánh dấu đoạn thời gian cần xử lý, sau đó backend trích xuất các khung hình tương ứng và chạy pipeline nhận dạng trên cửa sổ thời gian đã chọn. *Xem kết quả và minh chứng nhận dạng* cho phép người dùng xem lại kết quả của từng phiên xử lý, bao gồm danh sách biển số nhận dạng được, các xe bị từ chối kèm lý do và ảnh crop làm minh chứng. *Xem lịch sử nhận dạng* cung cấp danh sách các phiên xử lý đã thực hiện theo tài khoản đăng nhập, giúp người dùng tra cứu lại kết quả của các video hoặc trích đoạn đã xử lý trước đó.

0.1.2 Thiết kế kiến trúc

Lựa chọn kiến trúc phần mềm Hệ thống áp dụng kiến trúc **ba lớp** (Three-tier Architecture) kết hợp với mô hình Client–Server. Kiến trúc này phân tách ứng dụng thành ba tầng độc lập: tầng trình bày chịu trách nhiệm hiển thị giao diện và tương tác với người dùng, tầng xử lý nghiệp vụ thực thi toàn bộ logic ứng dụng, và tầng dữ liệu quản lý việc lưu trữ và truy vấn dữ liệu bền vững. Mỗi tầng chỉ giao tiếp với tầng liền kề thông qua giao diện được định nghĩa rõ ràng, giúp các tầng có thể thay đổi hoặc mở rộng độc lập mà không ảnh hưởng lẫn nhau.

Kiến trúc này được lựa chọn vì hệ thống có đặc thù xử lý nặng ở phía máy chủ (do sử dụng pipeline AI đa tầng với nhiều mô hình học sâu hoạt động nối tiếp), trong khi giao diện người dùng cần phản hồi nhanh và không bị chặn trong quá trình chờ kết quả. Việc tách biệt rõ tầng nghiệp vụ cũng cho phép cải tiến hay thay thế các mô hình AI mà không cần thay đổi giao diện hay cơ sở dữ liệu.



Hình 0.2: Biểu đồ phụ thuộc gói

Biểu đồ phụ thuộc gói

Tầng trình bày (Presentation Tier) Tầng trình bày gồm ba package. Package *Authentication* chứa các màn hình xác thực người dùng, bao gồm màn hình đăng nhập và thành phần bảo vệ định tuyến, ngăn người dùng chưa xác thực truy cập các trang nghiệp vụ. Package *MainView* tổ chức toàn bộ giao diện sau đăng nhập, gồm trang tổng quan, màn hình tải video, màn hình xem kết quả và minh chứng, màn hình lịch sử và màn hình chọn trích đoạn từ nguồn video. Package *APIClient* đóng vai trò cầu nối duy nhất giữa tầng trình bày và tầng nghiệp vụ, cung cấp hai thành phần: client xử lý các yêu cầu REST đồng bộ và client nhận kết quả theo luồng thời gian thực từ server.

Tầng nghiệp vụ (Business Logic Tier) Tầng nghiệp vụ gồm hai package. Package *WebAPI* tiếp nhận và xử lý toàn bộ yêu cầu HTTP từ client, được tổ chức thành bốn router theo chức năng: *AuthAPI* quản lý đăng ký, đăng nhập và xác thực JWT; *RecognitionAPI* tiếp nhận video đầu vào, khởi động pipeline xử lý và phát kết quả về client qua SSE; *MonitorAPI* quản lý các phiên chọn trích đoạn từ camera trực tiếp hoặc video tải lên, đồng thời xử lý các sự kiện được người dùng đánh dấu;; *HistoryAPI* cung cấp dữ liệu lịch sử phiên nhận dạng theo tài khoản. Package *PipelineALPR* thực thi toàn bộ luồng xử lý AI, gồm năm sub-package hoạt động

theo chuỗi: FrameProcessing giải mã và tiền xử lý khung hình từ video đầu vào; VehicleDetection phát hiện phương tiện trong từng khung hình; VehicleTracking theo dõi từng phương tiện xuyên suốt các khung hình liên tiếp; PlateDetection phát hiện và trích xuất vùng biển số; PlateOCR nhận dạng ký tự biển số; ResultAggregation tổng hợp kết quả từ nhiều khung hình để đưa ra kết quả cuối cùng cho từng phương tiện.

Tầng dữ liệu (Data Tier) Tầng dữ liệu gồm hai package với trách nhiệm phân tách rõ ràng. Package Database lưu trữ toàn bộ dữ liệu có cấu trúc, gồm UserStore (thông tin tài khoản và phiên xác thực), RecognitionStore (kết quả các phiên nhận dạng và bản ghi từng phương tiện) và MonitorEventStore lưu thông tin các sự kiện do người dùng đánh dấu trên nguồn video, gồm nguồn dữ liệu, khoảng thời gian được chọn, trạng thái xử lý và kết quả nhận dạng. Package ObjectStorage lưu trữ dữ liệu nhị phân, cụ thể là EvidenceImageStore chứa ảnh crop biển số và ảnh minh chứng phương tiện; package Database chỉ lưu URL tham chiếu đến các tài nguyên này thay vì nhúng trực tiếp ảnh vào document.

0.2 Xây dựng ứng dụng

0.2.1 Công nghệ sử dụng

Hệ thống được phát triển bằng các công cụ, ngôn ngữ lập trình và thư viện trong Bảng 1.

Bảng 1: Danh sách thư viện và công cụ sử dụng

Mục đích	Công cụ/Thư viện	Phiên bản	Địa chỉ URL
IDE lập trình	Visual Studio Code	1.104.0	https://code.visualstudio.com/
Quản lý mã nguồn	Git	2.43.0	https://git-scm.com/
Đóng gói ứng dụng	Docker	28.5.1	https://www.docker.com/
Ngôn ngữ backend	Python	3.10	https://www.python.org/
Runtime frontend	Node.js	20	https://nodejs.org/
Package manager	npm	11.6.3	https://www.npmjs.com/

Mục đích	Công cụ/Thư viện	Phiên bản	Địa chỉ URL
Backend API	FastAPI	$\geq 0.111.0$	https://fastapi.tiangolo.com/
ASGI server	Uvicorn	$\geq 0.29.0$	https://www.uvicorn.org/
Học sâu	PyTorch	2.4.1	https://pytorch.org/
Xử lý ảnh	OpenCV Python	4.10.0.84	https://opencv.org/
Phát hiện đối tượng	Ultralytics YOLO	$\geq 8.0.0$	https://docs.ultralytics.com/
Theo dõi đối tượng	BoxMOT	19.0.0	https://github.com/mikel-brostrom/boxmot
Suy luận ONNX	ONNX Runtime	1.20.0	https://onnxruntime.ai/
Tính toán ma trận	NumPy	1.26.4	https://numpy.org/
Tính toán khoa học	SciPy	$\geq 1.11.0$	https://scipy.org/
Huấn luyện mô hình	Lightning	2.x	https://lightning.ai/
Xử lý ảnh phụ trợ	Pillow	$\geq 10.0.0$	https://python-pillow.org/
Đọc cấu hình	PyYAML	$\geq 6.0.1$	https://pyyaml.org/
Xác thực mật khẩu	bcrypt	$\geq 4.1.0$	https://github.com/pyca/bcrypt
JWT	PyJWT	$\geq 2.8.0$	https://pyjwt.readthedocs.io/
Cơ sở dữ liệu	MongoDB	7	https://www.mongodb.com/
Driver goDB	Motor	$\geq 3.3.0$	https://motor.readthedocs.io/

Mục đích	Công cụ/Thư viện	Phiên bản	Địa chỉ URL
Lưu trữ ảnh	Supabase	$\geq 2.5.0$	https://supabase.com/
Frontend framework	React	18.3.1	https://react.dev/
Điều hướng frontend	React Router DOM	7.17.0	https://reactrouter.com/
Build frontend	Vite	5.4.21	https://vite.dev/
Thiết kế giao diện	Tailwind CSS	3.4.19	https://tailwindcss.com/
Streaming video	MediaMTX	v1.19.1	https://github.com/bluenviron/mediamtx
Xử lý video	FFmpeg	6.1.1	https://ffmpeg.org/
Kiểm thử	pytest	–	https://docs.pytest.org/

0.2.2 Frontend

Frontend của hệ thống là ứng dụng web một trang được xây dựng bằng React và Vite. Giao diện không thực hiện suy luận mô hình trực tiếp mà đóng vai trò tiếp nhận đầu vào từ người dùng, gửi yêu cầu tới backend, nhận tiến độ xử lý qua cơ chế streaming và hiển thị kết quả nhận dạng theo từng phương tiện hoặc từng sự kiện.

Ứng dụng được tổ chức theo hai chế độ chính. Chế độ thứ nhất xử lý toàn bộ một video được tải lên, phù hợp với nhu cầu đánh giá kết quả nhận dạng trên một tệp video hoàn chỉnh. Chế độ thứ hai cho phép người dùng chọn một trích đoạn cần xử lý từ nguồn camera trực tiếp hoặc từ video đã tải lên. Người dùng chuyển đổi giữa hai chế độ này thông qua thành phần điều hướng trên thanh tiêu đề.

Thành phần	Vai trò
Quản lý xác thực	Quản lý trạng thái đăng nhập, gọi các API xác thực và tự động gắn token bảo mật cho các yêu cầu ghi dữ liệu.
Màn hình làm việc chính	Điều phối trạng thái chính: file video, tiến độ xử lý, danh sách phương tiện hợp lệ và danh sách kết quả bị loại.
Bảng điều khiển nguồn	Cho phép người dùng chọn file video và các tùy chọn xử lý trước khi gửi yêu cầu lên server.
Khung hiển thị video	Hiển thị video gốc hoặc khung hình đã chú thích được server gửi về trong quá trình xử lý.
Bảng kết quả nhận dạng	Hiển thị kết quả theo từng phương tiện, gồm biểu số tổng hợp, độ tin cậy, ảnh minh chứng và các khung hình trong bộ đệm theo dõi.
Màn hình lịch sử	Truy vấn và hiển thị danh sách các phiên xử lý đã lưu và kết quả chi tiết của từng phiên.
Màn hình chọn trích đoạn	Quản lý các phiên chọn trích đoạn từ nguồn camera trực tiếp hoặc video tải lên, tiếp nhận lệnh đánh dấu sự kiện và hiển thị kết quả nhận dạng theo luồng.

Bảng 2: Các thành phần giao diện chính

Trong chế độ xử lý toàn bộ video, người dùng chọn file tại bảng điều khiển nguồn. Sau khi gửi yêu cầu, hệ thống nhận về một mã định danh công việc và mở kết nối streaming để nhận các sự kiện: tiến độ theo frame, khung hình đang xử lý, kết quả từng phương tiện và trạng thái hoàn thành. Dữ liệu nhận được được cập nhật vào trạng thái của màn hình chính, nhờ vậy giao diện hiển thị tiến độ và kết quả mà không cần tải lại trang.

Trong chế độ chọn trích đoạn, người dùng trước hết chọn nguồn dữ liệu, có thể là camera trực tiếp hoặc video tải lên. Khi người dùng đánh dấu một khoảng thời gian cần xử lý, frontend gửi thông tin phiên, loại nguồn và mốc thời gian đã chọn tới server. Backend sau đó trích xuất các khung hình thuộc khoảng thời gian này, chạy pipeline nhận dạng biểu số và đẩy tiến độ cùng kết quả về frontend theo cơ chế streaming. Cách tổ chức này giúp thao tác chọn đoạn trên giao diện diễn ra nhanh, trong khi phần xử lý AI được thực hiện bất đồng bộ ở phía server.

0.2.3 Backend

Backend được xây dựng bằng FastAPI với một điểm khởi động duy nhất chịu trách nhiệm định nghĩa route, kiểm tra đầu vào, quản lý hàng đợi streaming và điều phối các công việc xử lý. Các phần tính toán nặng được tách riêng thành module lõi gồm nạp mô hình, phát hiện phương tiện, phát hiện biểu số, theo dõi đối tượng,

nhận dạng ký tự, chấm điểm chất lượng và tổng hợp kết quả.

Khi ứng dụng khởi động, các mô hình AI được nạp một lần duy nhất, kết nối cơ sở dữ liệu được khởi tạo và tác vụ dọn dẹp các phiên chọn trích đoạn tạm thời được kích hoạt. Cách tổ chức này giúp tránh việc mỗi yêu cầu xử lý lại phải khởi tạo toàn bộ mô hình từ đầu, đồng thời đảm bảo các file video tạm và phiên không còn sử dụng được giải phóng sau một khoảng thời gian nhất định.

Nhóm API	Chức năng
Xác thực	Tạo tài khoản, đăng nhập, đăng xuất, kiểm tra phiên người dùng và cấp token bảo mật cho các yêu cầu ghi dữ liệu.
Xử lý video tải lên	Tiếp nhận video, tạo công việc xử lý bất đồng bộ, phát tiến độ và kết quả theo luồng thời gian thực, cung cấp khung hình preview và kết quả chi tiết của từng phương tiện.
Lịch sử nhận dạng	Liệt kê các phiên xử lý đã lưu, lấy thông tin tổng quan một phiên và danh sách kết quả nhận dạng; chỉ trả dữ liệu thuộc về người dùng hiện tại.
Chọn trích đoạn và sự kiện	Tạo và quản lý phiên chọn trích đoạn từ file tải lên hoặc nguồn camera trực tiếp, tiếp nhận lệnh đánh dấu sự kiện, xử lý đoạn video tương ứng ở backend, phát tiến độ và kết quả theo luồng, đồng thời cung cấp API tra cứu lại các sự kiện đã lưu.

Bảng 3: Các nhóm API chính của backend

Luồng xử lý video được thiết kế theo cơ chế bất đồng bộ. Backend kiểm tra định dạng, dung lượng và tính hợp lệ của file đầu vào, sau đó sinh mã định danh công việc, tạo hàng đợi kết quả và khởi chạy pipeline trong một luồng riêng. Trong quá trình xử lý, backend đẩy liên tục các bản tin gồm tiến độ theo frame, khung hình preview, kết quả từng phương tiện và trạng thái hoàn thành theo chuẩn SSE, nhờ đó giao diện có thể cập nhật gần thời gian thực mà không cần hỏi vòng lặp.

Về bảo mật, mật khẩu người dùng được băm bằng thuật toán bcrypt, phiên đăng nhập được lưu phía máy chủ và token xác thực được gửi về trình duyệt trong cookie bảo mật chỉ đọc qua HTTP. Các yêu cầu ghi dữ liệu trong luồng xử lý video chính yêu cầu thêm token chống giả mạo yêu cầu trong header. Cách triển khai này tách dữ liệu xác thực khỏi mã nguồn giao diện và tránh lưu token truy cập trong bộ nhớ trình duyệt.

Lớp lưu trữ được tách khỏi lớp xử lý nghiệp vụ. Cơ sở dữ liệu lưu thông tin người dùng, phiên xử lý, kết quả nhận dạng và sự kiện được đánh dấu. Object storage lưu

ảnh minh chứng để tránh lưu dữ liệu ảnh trực tiếp trong document. Nhờ cách tách này, backend API giữ vai trò điều phối, còn dữ liệu lớn và metadata được lưu ở các thành phần phù hợp.

0.2.4 Thiết kế cơ sở dữ liệu

Hệ thống sử dụng MongoDB làm cơ sở dữ liệu chính và một dịch vụ object storage tương thích S3 làm kho lưu trữ ảnh minh chứng. MongoDB lưu toàn bộ dữ liệu có cấu trúc gồm tài khoản người dùng, phiên đăng nhập, phiên xử lý video, kết quả nhận dạng và các sự kiện được đánh dấu trong chế độ chọn trích đoạn. Ảnh crop phương tiện, ảnh crop biển số và các khung hình minh chứng được lưu trên object storage; trong MongoDB chỉ lưu URL trỏ tới các đối tượng này.

Thiết kế này bám theo cách dữ liệu được sinh ra trong pipeline: một phiên xử lý có thể sinh nhiều phương tiện, mỗi phương tiện có một biển số cuối cùng và nhiều crop theo thời gian. Nếu lưu ảnh nhị phân trực tiếp trong document, kích thước tăng nhanh và khó mở rộng. Vì vậy hệ thống tách metadata khỏi ảnh: cơ sở dữ liệu giữ khóa truy vấn, kết quả OCR và các URL; object storage giữ dữ liệu ảnh.

Các collection chính Bảng 4 mô tả ánh xạ giữa các thực thể nghiệp vụ và collection trong MongoDB.

Bảng 4: Các collection và document trong MongoDB

Thực thể	Collection	Mô tả
Tài khoản người dùng	users	Lưu thông tin đăng ký, email duy nhất, mật khẩu đã băm và trạng thái tài khoản.
Phiên đăng nhập	auth_sessions	Lưu phiên đăng nhập phía máy chủ; tham chiếu đến tài khoản người dùng và hỗ trợ thu hồi phiên.
Phiên nhận dạng	recognition_sessions	Lưu metadata của một lượt xử lý video: tên file nguồn, trạng thái, số frame và chế độ xử lý.
Kết quả nhận dạng	recognition_records	Lưu kết quả cuối cùng của từng phương tiện: biển số tổng hợp, độ tin cậy, lớp xe và URL ảnh minh chứng. Mỗi record nhúng thêm danh sách crop biển số theo thời gian phục vụ màn hình xem minh chứng.
Sự kiện video	events	Lưu một cửa sổ thời gian được người dùng đánh dấu trên nguồn video, gồm loại nguồn, khoảng thời gian, trạng thái xử lý và danh sách phương tiện phát hiện được nhúng trực tiếp trong document.
Ảnh minh chứng	(Object Storage)	Không phải collection MongoDB. File ảnh lưu trên object storage; MongoDB chỉ lưu URL tham chiếu trong các trường tương ứng của kết quả nhận dạng và sự kiện.

Thiết kế chỉ mục Bảng 5 liệt kê các chỉ mục chính được tạo để phục vụ các truy vấn thường gặp của ứng dụng.

Bảng 5: Các chỉ mục MongoDB chính

Collection	Chỉ mục	Mục đích
users	email (duy nhất); created_at giảm dần	Đảm bảo email không trùng và tăng tốc đăng nhập.
auth_sessions	session_id (duy nhất); user_id; expires_at	Kiểm tra và truy vết phiên đăng nhập theo người dùng.
recognition_sessions	session_id (duy nhất); (user_id, created_at); status	Lấy phiên theo mã, liệt kê lịch sử gần nhất và lọc theo trạng thái.
recognition_records	session_id; (session_id, track_id) duy nhất; plate_text; vehicle_class	Lấy kết quả theo phiên, tránh lưu trùng track và hỗ trợ tìm kiếm biển số.
events	event_id (duy nhất); session_id; marked_at giảm dần; status	Lấy chi tiết sự kiện, liệt kê theo phiên và theo thời gian đánh dấu.

0.3 Triển khai

Ứng dụng được triển khai theo mô hình tách riêng frontend, backend xử lý AI và các dịch vụ lưu trữ. Frontend được triển khai trên Vercel, backend được đóng gói bằng Docker và chạy trên RunPod GPU. Cơ sở dữ liệu và lưu trữ ảnh kết quả sử dụng các dịch vụ bên ngoài để giảm tải cho server xử lý chính.

Thành phần	Cấu hình triển khai
Frontend	React/Vite, triển khai trên Vercel
Backend API	FastAPI, chạy trên cổng 8000
Server xử lý AI	RunPod GPU, NVIDIA RTX 3090
Môi trường GPU	PyTorch 2.4.1, CUDA 12.1, cuDNN 9
Streaming video	MediaMTX, sử dụng WebRTC/RTSP
Cơ sở dữ liệu	MongoDB Atlas
Lưu trữ ảnh kết quả	Supabase Storage
Giới hạn dung lượng upload	512 MB

Bảng 6: Cấu hình triển khai ứng dụng

Trong môi trường thử nghiệm cục bộ, hệ thống được chạy bằng Docker Compose gồm bốn dịch vụ chính: backend API, frontend web, MediaMTX và MongoDB. Backend mở cổng 8000, frontend mở cổng 3000, MongoDB mở cổng 27017, MediaMTX sử dụng các cổng 8554, 8889, 8189/udp và 9997. Cấu hình này phục vụ kiểm thử chức năng đăng nhập, tải video, xử lý nhận dạng và xem kết quả

trực tiếp.

Trong môi trường triển khai thử nghiệm trên server GPU, backend và MediaMTX được đóng gói trong cùng một Docker image. Backend đảm nhiệm các bước phát hiện phương tiện, phát hiện biển số, tracking, OCR và trả kết quả qua API. MediaMTX phục vụ luồng video trực tiếp, hỗ trợ hiển thị nguồn camera trong chế độ chọn trích đoạn trên giao diện web. Các mô hình như detector phương tiện, detector biển số, OCR và ReID được đặt trên network volume của RunPod.